

Project 1: Automated Port Scanner + Banner Grabbing

Goal

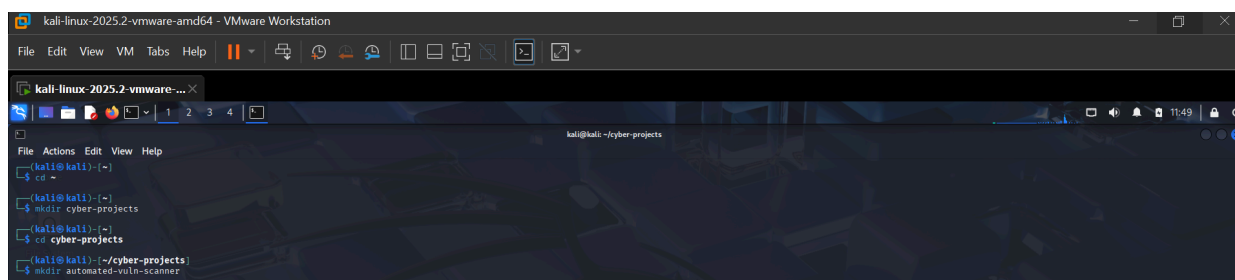
Build a Python script that scans a target IP/domain for open ports and attempts banner grabbing to identify services running.

Steps:

1. Set up Project Folder

We organized our workspace in Kali Linux:

```
cd ~  
mkdir cyber-projects  
cd cyber-projects  
mkdir automated-vuln-scanner  
cd automated-vuln-scanner
```



This gave us a clean folder for our first project.

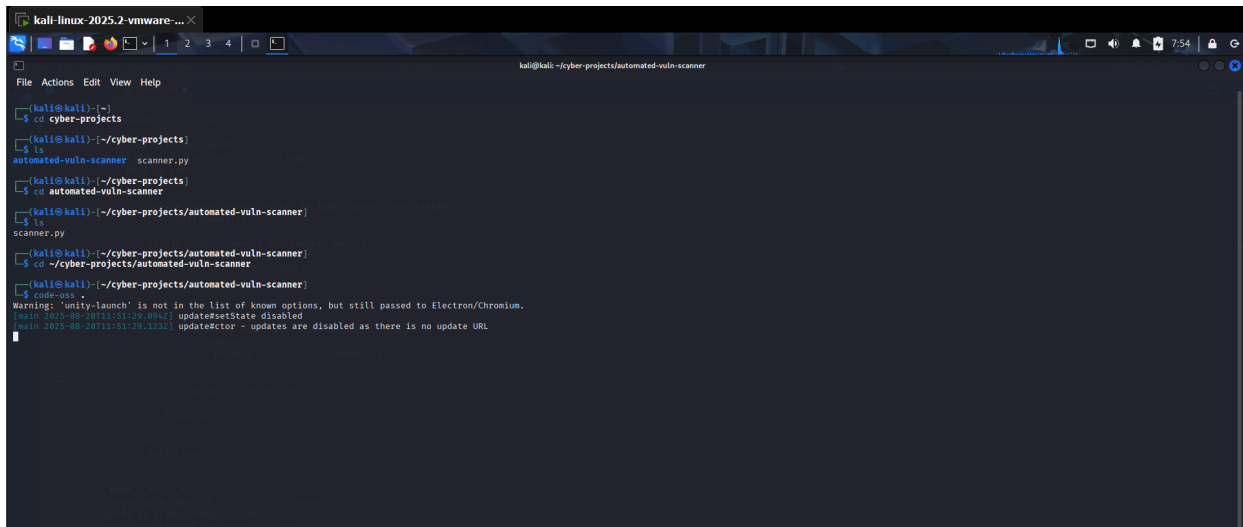
2. Installed VS Code (code-oss)

Since Kali doesn't come with VS Code by default, we installed it:

```
sudo apt update  
sudo apt install code-oss -y
```

Opened our project folder in VS Code:

```
cd ~/cyber-projects/automated-vuln-scanner  
code-oss .
```



3. Created Python File

Inside VS Code:

- New File → `scanner.py`

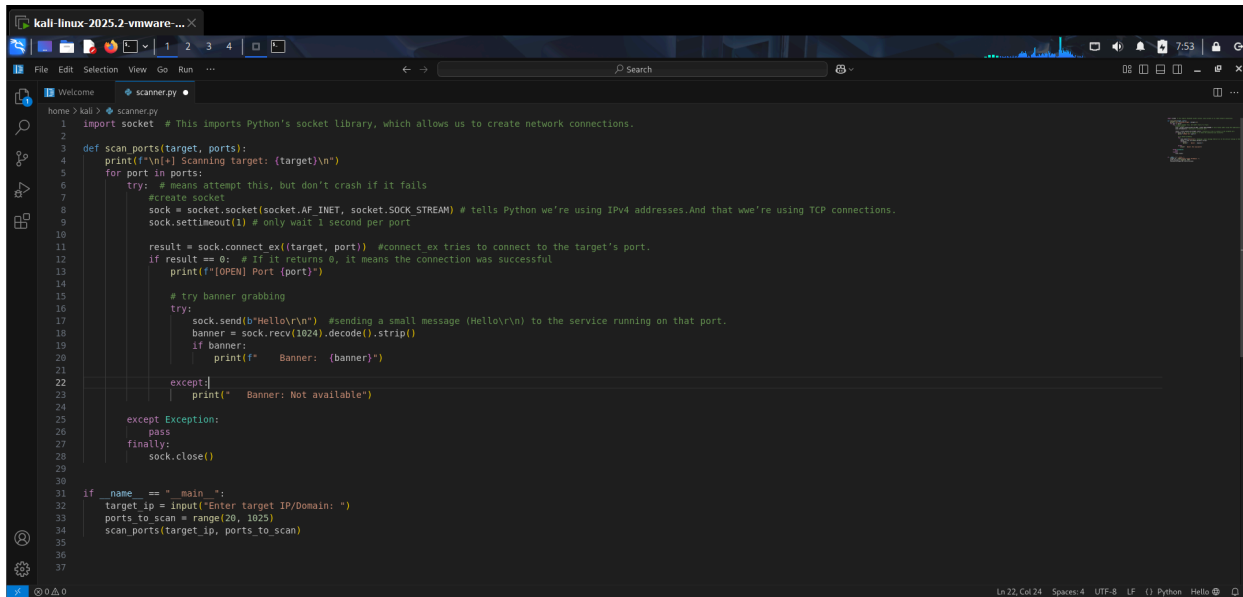
4. Wrote the Script

We built the scanner step by step:

- Used Python `socket` module to check ports.
- Added timeout (1 second per port).
- Printed `[OPEN] Port <number>` when open.

- Implemented **banner grabbing** by sending `Hello\r\n` and trying to capture the response.
- Added exception handling so the script doesn't crash.

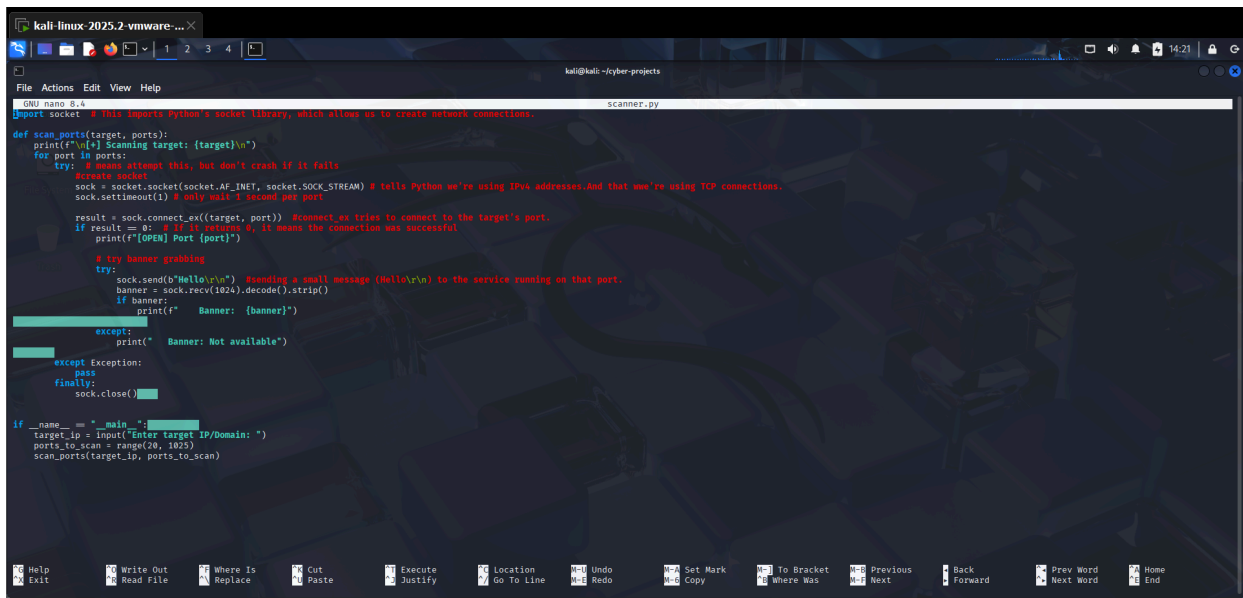
Final code:



```

1 import socket # This imports Python's socket library, which allows us to create network connections.
2
3 def scan_ports(target, ports):
4     print(f'\n[+] Scanning target: {target}\n')
5     for port in ports:
6         try: # means attempt this, but don't crash if it fails
7             #create socket
8             sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # tells Python we're using IPv4 addresses. And that we're using TCP connections.
9             sock.settimeout(1) # only wait 1 second per port
10
11             result = sock.connect_ex((target, port)) #connect_ex tries to connect to the target's port.
12             if result == 0: # if it returns 0, it means the connection was successful
13                 print(f'[OPEN] Port {port}')
14
15                 # try banner grabbing
16                 try:
17                     sock.send(b'Hello\r\n') #sending a small message (Hello\r\n) to the service running on that port.
18                     banner = sock.recv(1024).decode().strip()
19                     if banner:
20                         print(f'Banner: {banner}')
21
22                 except:
23                     print("Banner: Not available")
24
25         except Exception:
26             pass
27         finally:
28             sock.close()
29
30
31 if __name__ == "__main__":
32     target_ip = input("Enter target IP/Domain: ")
33     ports_to_scan = range(20, 1025)
34     scan_ports(target_ip, ports_to_scan)
35
36
37

```



```

1 import socket # This imports Python's socket library, which allows us to create network connections.
2
3 def scan_ports(target, ports):
4     print(f'\n[+] Scanning target: {target}\n')
5     for port in ports:
6         try: # means attempt this, but don't crash if it fails
7             #create socket
8             sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # tells Python we're using IPv4 addresses. And that we're using TCP connections.
9             sock.settimeout(1) # only wait 1 second per port
10
11             result = sock.connect_ex((target, port)) #connect_ex tries to connect to the target's port.
12             if result == 0: # if it returns 0, it means the connection was successful
13                 print(f'[OPEN] Port {port}')
14
15                 # try banner grabbing
16                 try:
17                     sock.send(b'Hello\r\n') #sending a small message (Hello\r\n) to the service running on that port.
18                     banner = sock.recv(1024).decode().strip()
19                     if banner:
20                         print(f'Banner: {banner}')
21
22                 except:
23                     print("Banner: Not available")
24
25         except Exception:
26             pass
27         finally:
28             sock.close()
29
30
31 if __name__ == "__main__":
32     target_ip = input("Enter target IP/Domain: ")
33     ports_to_scan = range(20, 1025)
34     scan_ports(target_ip, ports_to_scan)
35
36
37

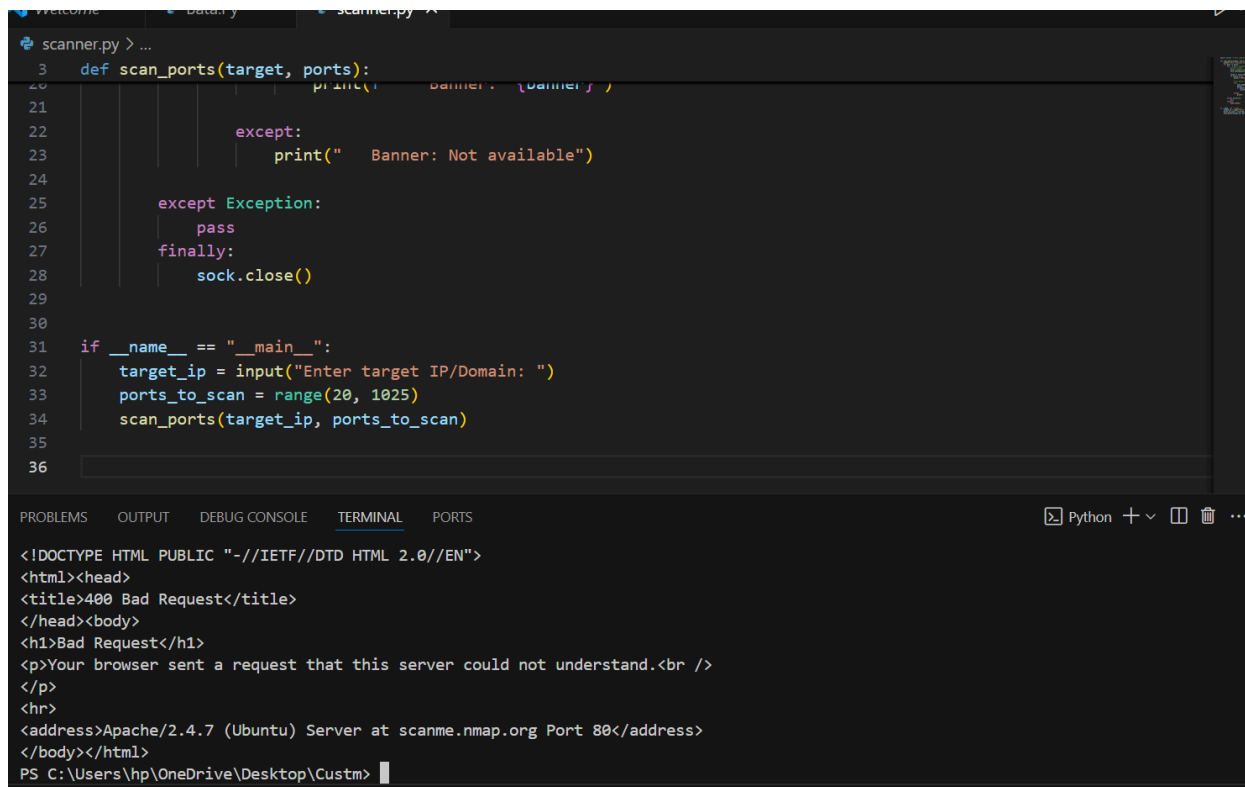
```

5. Ran the Script

```
cd ~/cyber-projects/automated-vuln-scanner
python3 scanner.py
```

Enter target IP/Domain: scanme.nmap.org

```
scanner.py > scan_ports
1 import socket # This imports Python's socket library, which allows us to create network connections.
2
3 def scan_ports(target, ports):
4     print(f"\n[+] Scanning target: {target}\n")
5     for port in ports:
6         try: # means attempt this, but don't crash if it fails
7             #create socket
8             sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # tells Python we're using IPv4 addresses. And th
9             sock.settimeout(1) # only wait 1 second per port
10
11             result = sock.connect_ex((target, port)) #connect_ex tries to connect to the target's port.
12             if result == 0: # If it returns 0, it means the connection was successful
13                 print(f"[OPEN] Port {port}")
14
15                 # try banner grabbing
16                 try:
17                     sock.send(b"Hello\r\n") #sending a small message (Hello\r\n) to the service running on that port
18                     banner = sock.recv(1024).decode().strip()
```

The image shows a VS Code editor window with a file named `scanner.py`. The script defines a `scan_ports` function that takes a `target` and a list of `ports` as arguments. It uses a `socket` module to connect to each port and retrieve a banner. The script includes error handling for connection failures and a `finally` block to close the socket. The main block prompts the user for a target IP/domain and a range of ports to scan (20 to 1025).

```
scanner.py > ...
3 def scan_ports(target, ports):
20     print(f"Banner: {banner}")
21
22     except:
23         print("Banner: Not available")
24
25     except Exception:
26         pass
27     finally:
28         sock.close()
29
30
31 if __name__ == "__main__":
32     target_ip = input("Enter target IP/Domain: ")
33     ports_to_scan = range(20, 1025)
34     scan_ports(target_ip, ports_to_scan)
35
36
```

The terminal output shows the result of running the script on `scanme.nmap.org`. It displays a 400 Bad Request error from the Apache server.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + - [ ] [ ] ...

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>400 Bad Request</title>
</head><body>
<h1>Bad Request</h1>
<p>Your browser sent a request that this server could not understand.<br />
</p>
<hr>
<address>Apache/2.4.7 (Ubuntu) Server at scanme.nmap.org Port 80</address>
</body></html>
PS C:\Users\hp\OneDrive\Desktop\Custm>
```

6. Example Output

The script found open ports and tried to grab banners:

[+] Scanning target: scanme.nmap.org

[OPEN] Port 22

Banner: SSH-2.0-OpenSSH_6.6.1p1 Ubuntu-2ubuntu2.13

[OPEN] Port 80

Banner: HTTP/1.1 400 Bad Request

What We Achieved

- Built a **Python port scanner** that successfully detects open ports on any IP or domain.

- Implemented **banner grabbing**, allowing the tool to identify running services such as SSH and HTTP.
 - Demonstrated real-world results by scanning scanme.nmap.org, showing open ports and detailed server information.
 - Gained **hands-on experience with socket programming** and network protocols.
 - Learned to structure and run Python projects efficiently in **Kali Linux using VS Code**.
 - Strengthened **problem-solving skills** through handling errors and exceptions gracefully.
 - Laid the groundwork for extending the tool into an **Automated Vulnerability Checker** that could cross-reference services with known CVEs.
-